```
import os
import pandas as pd
import numpy as np
import warnings
import matplotlib.pyplot as plt
import time
from sklearn.metrics import precision_score, recall_score, f1_score
from TSB_UAD.models.sand import SAND
warnings.filterwarnings('ignore')
from sklearn.preprocessing import MinMaxScaler
from TSB_UAD.models.sand import SAND
from sklearn.metrics import roc_auc_score,classification_report
warnings.filterwarnings("ignore", category=FutureWarning)
```

Get dataset from csv

```
def load_data(file_path):
    """
    Load time series and labels from a .out file with comma-separated values.
    Assumes format: measurement,anomaly
    """

    # Read CSV normally so pandas uses the header row
    df = pd.read_csv(file_path)

    # Extract columns correctly
    X = df["measurement"].values.astype(np.float32)
    y = df["anomaly"].values.astype(int)

    return X, y
```

Sand for streaming

```
file_path = "normality1.csv"
# file_path = "normality2.csv"
# file_path = "normality3.csv"
# file_path = "normality4.csv"
data, label = load_data(file_path)

slidingWindow = 32

batch_time = 0
all_precisions = []
all_recalls = []
all_f1s = []
```

```
threshold = 0.5

# Prepare data for semisupervised method.
# Here, the training ratio = 0.1
print("Estimated Subsequence length: ",slidingWindow)
print("Time series length: ",len(data))
print("Number of abnormal points: ",list(label).count(1))

clf =
SAND(pattern_length=slidingWindow,subsequence_length=4*(slidingWindow)
)
x = data
start_time = time.time()

# Normality 1
clf.fit(x.astype(np.float64),online=True,alpha=0.5,init_length=5000,ba
tch_size=500,verbose=True,overlaping_rate=int(4*slidingWindow))

# Rest of normalities
#
clf.fit(x.astype(np.float64),online=True,alpha=0.5,init_length=5000,ba
tch_size=5000,verbose=True,overlaping_rate=int(4*slidingWindow))

score = clf.decision_scores_
score =
MinMaxScaler(feature_range=(0,1)).fit_transform(score.reshape(-
1,1)).ravel()

end_time = time.time()
batch_time=end_time - start_time


print(f"runtime: {batch_time / 60:.4f} minutes")

# Thresholding scores to get binary predictions (you can adjust
threshold)
threshold = 0.5
preds = (score >= threshold).astype(int)

# Evaluate predictions
precision = precision_score(label[:len(preds)], preds)
recall = recall_score(label[:len(preds)], preds)
f1 = f1_score(label[:len(preds)], preds)
roc_auc = roc_auc_score(label[:len(preds)], preds)


# Evaluation
print(" Evaluate predictions:")
print(f"Precision: {precision:.4f}")
```

```python
print(f"Recall:    {recall:.4f}")
print(f"F1 Score:  {f1:.4f}")
print(f"ROC-AUC:   {roc_auc:.4f}")
print(classification_report(label[:len(preds)], preds, digits=4))

# Optionally, save to lists if you want to track over batches
all_precisions.append(precision)
all_recalls.append(recall)
all_f1s.append(f1)
```

```
Estimated Subsequence length:  32
Time series length:  1680
Number of abnormal points:  10
0-->[STOP]: score length 1680
runtime: 0.3796 minutes
 Evaluate predictions:
Precision: 0.0340
Recall:    1.0000
F1 Score:  0.0658
ROC-AUC:   0.9150
              precision    recall  f1-score   support

           0     1.0000    0.8299    0.9071      1670
           1     0.0340    1.0000    0.0658        10

    accuracy                         0.8310      1680
   macro avg     0.5170    0.9150    0.4864      1680
weighted avg     0.9943    0.8310    0.9021      1680
```